Memento SQL Objet

Types.

```
CREATE TYPE telephone AS OBJECT (
    typet VARCHAR2(50), --perso ou pro
    numero CHAR(10)
);

CREATE TYPE clientT AS OBJECT (
    nomClient VARCHAR2(30),
    telephoneClient TELEPHONE
);

CREATE TABLE client OF CLIENTT (
    CONSTRAINT ck_tel CHECK (telephoneClient.typet in ('perso', 'pro'))
);

INSERT INTO client VALUES('Dupond', telephone('pro','0187654321')));
SELECT nomclient, c.telephoneclient.numero FROM client c;
```

Méthodes.

```
CREATE OR REPLACE TYPE personnelT AS OBJECT (
    idpers NUMBER(2),
    MEMBER FUNCTION anciennete RETURN number,
    MEMBER PROCEDURE majSalaire
);

CREATE OR REPLACE TYPE BODY personnelT AS
    MEMBER FUNCTION anciennete RETURN NUMBER IS
    BEGIN
        RETURN (trunc((sysdate - dateEmb)/365));
    END;
    MEMBER PROCEDURE majSalaire IS
    BEGIN
        if anciennete() > 10 THEN
            salaire := salaire * 2;
        END IF;
    END;
END;

SELECT nomPers, p.anciennete() FROM personnel p;

DECLARE
    pers personnelt;
    CURSOR cpers IS SELECT idpers FROM personnel;
BEGIN
    FOR cpers_rec IN cpers LOOP
        --VALUE : pour recuperer l'objet "ligne"
        SELECT value(p) into pers FROM personnel p WHERE cpers_rec.idpers = idpers;
        --modification de l'objet par appel a la methode
        pers.majsalaire();
        -- mise a jour de l'objet dans la table
        UPDATE personnel p SET value(p) = pers WHERE cpers_rec.idpers = idpers;
    END LOOP;
END;
```

Héritage.

```
CREATE OR REPLACE TYPE personnelT AS OBJECT (
    idpers NUMBER(2),
) NOT FINAL; -- pour classe abstraite, rajouter NOT INSTANTIABLE

CREATE OR REPLACE TYPE vendeurT UNDER personnelT (); --INSTANTIABLE/FINAL par defaut
CREATE OR REPLACE TYPE livreurT UNDER personnelT (permis CHAR(1)); --INSTANTIABLE/FINAL par
    defaut
```

Collections.

```
  --nested table
CREATE OR REPLACE TYPE ingredientT AS OBJECT
(
    noming VARCHAR2(40)
);

CREATE OR REPLACE TYPE ingredientsT AS TABLE OF ingredientt;

CREATE OR REPLACE TYPE pizzaT AS OBJECT
(
    nompizza VARCHAR2(40),
    prix NUMBER(4, 2),
    ingr ingredientst
);

CREATE TABLE pizza OF pizzat
(
    CONSTRAINT pk_piz PRIMARY KEY (nompizza)
)
NESTED TABLE ingr STORE AS tabingr;

INSERT INTO pizza VALUES ('Reine', 12.5,
    ingredientst(ingredientt('Sauce tomate'),
                 ingredientt('Mozarella')));

--nombre de pizzas par ingredients
SELECT t.noming, count(*) FROM pizza p, TABLE(p.ingr) t GROUP BY t.noming;
INSERT INTO TABLE (SELECT p.ingr FROM pizza p WHERE nompizza = 'Margherita')
            VALUES ('Huile d''olive');
UPDATE pizza p SET prix = prix * 1.05
    WHERE EXISTS (SELECT * FROM TABLE (p.ingr) t WHERE t.noming = 'Jambon');
UPDATE TABLE (SELECT p.ingr FROM pizza p WHERE nompizza = 'Reine') ti
    SET ti.noming = 'Gruyere' WHERE ti.noming = 'Mozarella';

--varray
CREATE OR REPLACE TYPE taillesT as VARRAY (2) OF VARCHAR2(10);

CREATE OR REPLACE TYPE pizzaT AS OBJECT
(
    nompizza VARCHAR2(40),
    tailles taillest
);

INSERT INTO pizza VALUES ('Reine', taillest('grande', 'petite'));
--column_value pour acceder pour une colonne d'un VARRAY ou NESTED TABLE sans nom
SELECT nompizza FROM pizza p, TABLE(p.tailles) t WHERE t.column_value = 'grande';

--la manipulation des VARRAY se fait en PL/SQL
DECLARE
    ttailles taillest;
BEGIN
    ...
    FOR i IN 1..ttailles.count LOOP
        IF ttailles(i) NOT IN ('petite', 'grande') THEN
            raise_application_error(-20001, 'taille invalide');
        END IF;
    END LOOP;
END;
```